

ESPECIFICACION TECNICA UNIFICADA

Arquitectura de Base de Datos de Ultra Alto Rendimiento

MongoDB + DragonflyDB (Redis con esteroides)

Caché distribuida, streams de eventos, alta disponibilidad, cifrado de nivel empresarial y soporte a grandes cargas de datos – consolidados en una sola arquitectura de código.

Proyecto: Motor de datos de alto rendimiento – MongoDB + DragonflyDB

Organización: Magnus Norgaard

Documento: Propuesta de diseño de ingeniería (arquitectura consolidada)

Versión: 1.0

Naturaleza: Síntesis de 4 especificaciones fuente en una arquitectura única

Contenido

- 00** Resumen ejecutivo
- 01** Objetivos y principios de diseño
- 02** DragonflyDB: «Redis con esteroides»
- 03** Arquitectura general unificada
- 04** Componentes del sistema
- 05** Patrones de caché y flujos de operación
- 06** Arquitectura orientada a eventos – Streams
- 07** Escalabilidad
- 08** Alta disponibilidad y resiliencia
- 09** Durabilidad y persistencia
- 10** Seguridad – cifrado y control de acceso
- 11** Workers en segundo plano y respaldos
- 12** Índices
- 13** Despliegue
- 14** Arquitectura de código unificada
- 15** Características de rendimiento
- 16** Hoja de ruta (roadmap)
- 17** Consideraciones de ingeniería y decisiones
- 18** Conclusión

0 Resumen ejecutivo

Este documento consolida cuatro especificaciones técnicas independientes en una arquitectura única, coherente y lista para producción. El objetivo es construir el motor de datos más rápido, potente y tolerante a grandes cargas posible, combinando un almacén de registro persistente (MongoDB) con una capa de memoria de ultra alto rendimiento (DragonflyDB), y elevando ambos con las prácticas de seguridad, durabilidad y operación descritas en las fuentes.

Premisa central del diseño: «REDIS es DragonflyDB». DragonflyDB es un almacén en memoria compatible con el protocolo de Redis –un reemplazo directo (drop-in)– pero multihilo y con arquitectura moderna: «Redis con esteroides». Por eso, en toda esta arquitectura, donde las fuentes dicen «Redis», la implementación de referencia es DragonflyDB, conservando la totalidad del ecosistema de comandos, Streams, TTL y clientes de Redis.

Fuentes consolidadas

#	fuerce	Aporte principal a la arquitectura unificada
1	MONGO_README (Engineering Paper)	API Gateway en Rust, patrón Cache-Aside + Write-Through, TTL de 300 s, balanceo de carga, failover y despliegue en Kubernetes.
2	REDIS_README (Infraestructura)	Endpoint único (un solo connection string), proxy HAProxy frente al clúster, Docker Compose, cliente Rust y Streams.
3	REDIS.docx (Especificación)	Streams y Consumer Groups, 16 384 hash slots / sharding, replicación, Smart Clients, protocolo Gossip, failover automático, persistencia híbrida AOF+RDB, replay de eventos.
4	Requirements.pdf (Motor de datos)	Motor de almacenamiento cifrado, DragonflyDB como caché, cifrado AES-256-GCM, envelope encryption con Cloud KMS, WAL, workers en segundo plano, backups cifrados, índices B+ y hoja de ruta.

CONVENCIÓN Donde una capacidad proviene íntegramente de una fuente se conserva su semántica original. Donde la fusión introduce una decisión de diseño (p. ej. mapear el «motor de archivos cifrado» sobre MongoDB), se marca explícitamente como «Decisión de diseño» para su validación por los responsables técnicos antes del desarrollo.

1 Objetivos y principios de diseño

La arquitectura persigue simultáneamente rendimiento extremo y garantías de datos de nivel empresarial. Los objetivos, tomados y unificados de las cuatro fuentes, son:

- **Latencia mínima** – respuestas de milisegundos servidas desde memoria (DragonflyDB) para patrones de acceso repetidos.
- **Throughput masivo** – capacidad de sostener millones de consultas por segundo (QPS) mediante caché en memoria multihilo, sharding y réplicas de lectura.
- **Escalabilidad horizontal** – crecer por réplicas de API sin estado, shards de datos y miembros de replica set, sin rediseñar la aplicación.

- **Alta disponibilidad y failover automático** – autorreparación en milisegundos ante fallos de nodo, sin intervención manual.
- **Durabilidad y cero pérdida de datos** – persistencia híbrida AOF+RDB, WAL y replica set de MongoDB como sistema de registro autoritativo.
- **Seguridad de datos en reposo y en tránsito** – cifrado AES-256-GCM, envelope encryption con KMS, TLS, hashing Argon2id y RBAC con auditoría.
- **Consistencia de caché controlada** – invalidación por TTL y sincronización write-through, con write-behind opcional respaldado por WAL.
- **Simplicidad operativa** – un único connection string para la aplicación; la topología queda oculta tras el cliente/proxy.

Roles fundamentales

Componente	rol	naturaleza de los datos
MongoDB Replica Set	Sistema de registro (source of truth)	Datos permanentes, autoritativos
DragonflyDB (Redis-compatible)	Caché distribuida en memoria + bus de eventos (Streams)	Objetos temporales con TTL; log de eventos
API Gateway en Rust	Lógica de negocio, sin estado	Ninguno (stateless)
Cloud KMS	Custodia de la clave maestra (envelope encryption)	Claves cifradas, nunca en claro

2 DragonflyDB: «Redis con esteroides»

La decisión clave de esta arquitectura es adoptar DragonflyDB en el rol que las especificaciones fuente asignan a Redis. DragonflyDB implementa el mismo protocolo de cable (RESP) y la misma API de comandos que Redis –incluidos Streams, Consumer Groups, TTL/EX y estructuras de datos– por lo que actúa como reemplazo directo sin cambiar el código de aplicación ni los clientes.

La diferencia está en el motor: **Redis es de un solo hilo por instancia**; DragonflyDB es multihilo con una arquitectura shared-nothing que aprovecha todos los núcleos de una máquina. En la práctica, esto multiplica el throughput por instancia y reduce la complejidad operativa: mucho de lo que en Redis exige un clúster de 6 nodos, en DragonflyDB lo resuelve un solo nodo grande escalando verticalmente.

Equivalencia de capacidades

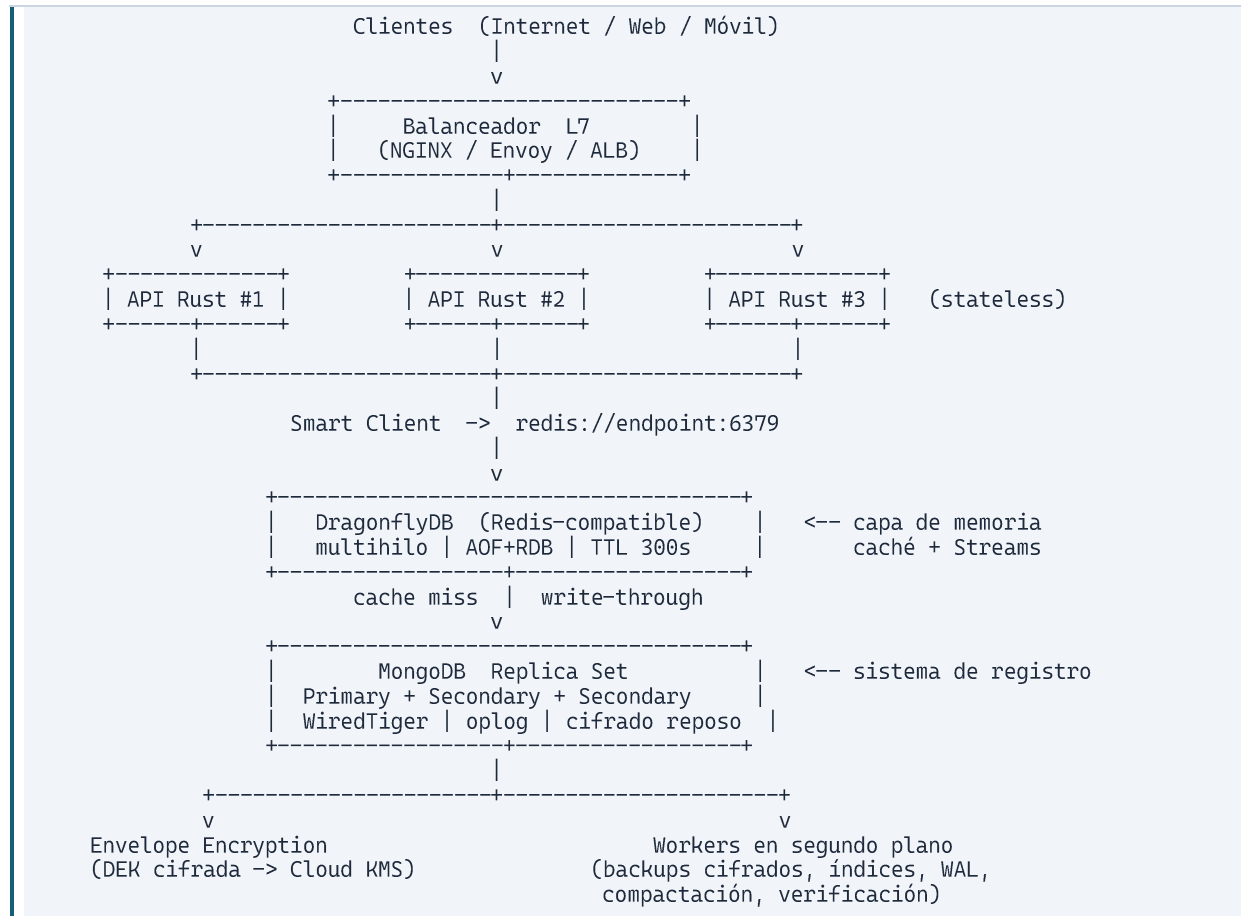
Capacidad de la especificación (redis)	En DragonflyDB
Protocolo y comandos (GET/SET/EX, etc.)	Compatible drop-in; mismos clientes (redis-rs, ioredis, redis-py...)
Redis Streams + Consumer Groups (XADD/XREADGROUP/XACK)	Soportado con la misma semántica de log inmutable y PEL
Persistencia (snapshots / append)	Snapshotting propio de alto rendimiento; compatible con RDB
TTL / expiración (SET ... EX 300)	Idéntico
Escalado a millones de QPS	Vertical multi-core en un nodo; menos piezas móviles que un clúster
Modo clúster (sharding)	Modo cluster emulado disponible si se requiere partición horizontal

VENTAJA CLAVE Con Redis Cluster, obtener «un solo connection string» exigía anteponer HAProxy a 6 nodos (fuente 2). Con DragonflyDB, un único proceso multihilo ya expone un solo endpoint que escala en núcleos: se conserva la simplicidad de `redis://endpoint:6379` y se elimina buena parte de la infraestructura intermedia.

A VALIDAR El modo cluster/particionado de DragonflyDB difiere operativamente de Redis Cluster. Si el volumen supera la RAM de un solo nodo, decidir entre (a) escalado vertical de un nodo Dragonfly de gran memoria o (b) modo cluster emulado. Decisión de diseño pendiente de confirmación.

3 Arquitectura general unificada

El sistema se organiza en cuatro planos: entrada y balanceo, cómputo sin estado (API Rust), memoria de ultra alto rendimiento (DragonflyDB) y registro durable (MongoDB), rodeados por servicios transversales de seguridad (KMS) y operación (workers en segundo plano).



Patrón de datos: Cache-Aside para lecturas y Write-Through para escrituras, con MongoDB como fuente autoritativa y DragonflyDB reflejando el estado más reciente con un TTL de 300 segundos (cinco minutos).

4 Componentes del sistema

4.1 Balanceador de carga (L7)

Punto de entrada único de tráfico HTTP. Responsabilidades:

- Distribuir peticiones entre las réplicas de API.
- Health checks y expulsión de instancias caídas.
- Terminación TLS/SSL.
- Reintento de peticiones fallidas.

4.2 API Gateway en Rust (sin estado)

Cada instancia de gateway es stateless, lo que habilita el autoescalado horizontal (p. ej. Horizontal Pod Autoscaler en Kubernetes). Se ejecutan tres réplicas simultáneas (API-1, API-2, API-3). Responsabilidades:

- Autenticación y autorización.
- Validación de peticiones.
- Lógica de negocio.
- Driver de MongoDB.
- Smart Client de Redis/DragonflyDB (*enrutamiento directo al nodo/slot correcto*).

4.3 DragonflyDB – capa de memoria (Redis-compatible)

Actúa como caché distribuida y como bus de eventos (Streams). Provee, con compatibilidad Redis:

- Sharding automático (cuando se usa modo clúster) sobre 16 384 hash slots.
- Replicación primary/replica para escalado de lectura y failover.
- Protocolo Gossip y failover automático (en modo clúster).
- Enrutamiento por Smart Client.
- Persistencia híbrida AOF + RDB.
- Streams inmutables con Consumer Groups.

4.4 MongoDB Replica Set – sistema de registro

MongoDB es la única fuente de datos permanentes. Configuración base: un Primary y dos Secondary. El Primary acepta escrituras; los Secondary sirven lecturas y quedan listos para elección. Sólo MongoDB almacena datos permanentes; DragonflyDB almacena objetos cacheados temporales.

APORTE NATIVO MongoDB (motor wiretiger) ya ofrece de fábrica muchas capacidades que la fuente 4 describía para un motor de archivos propio: índices B-tree, oplog (equivalente a un WAL), replica set con failover, y cifrado en reposo. La arquitectura unificada aprovecha estas capacidades nativas en lugar de reimplementarlas.

4.5 Motor de almacenamiento seguro (de la fuente 4)

La fuente 4 propone un motor de almacenamiento con cifrado por colección. En la arquitectura unificada, sus principios se aplican sobre MongoDB y sobre los backups, no como un segundo motor de disco paralelo:

- **Cifrado en reposo AES-256-GCM** de colecciones y respaldos.
- **Clave por colección (DEK)** bajo envelope encryption con Cloud KMS (ver §10).
- **Client-Side Field-Level Encryption (CSFLE)** para campos sensibles individuales cuando aplique.

4.6 Endpoint único (un solo connection string)

La aplicación conoce exclusivamente un endpoint. Con Redis Cluster clásico esto se lograba con HAProxy delante de los 6 nodos; con DragonflyDB, el propio nodo multihilo expone ese endpoint. En ambos casos, la cadena de conexión no cambia aunque crezca la topología.

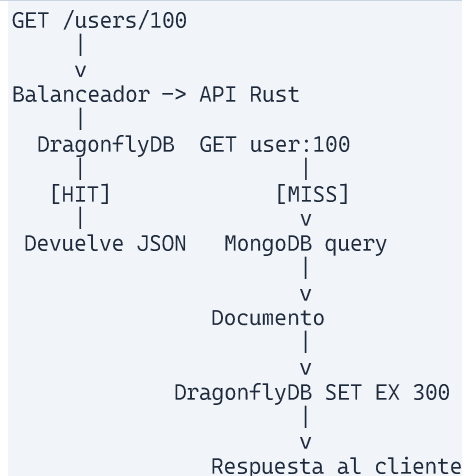
```
Aplicación -> redis://127.0.0.1:6379 (sin credenciales)
Aplicación -> redis://:password@127.0.0.1:6379 (con contraseña)
```


5 Patrones de caché y flujos de operación

5.1 Estrategia de caché

- **Cache-Aside (lecturas):** la aplicación consulta primero la caché; si hay miss, va a MongoDB y repuebla la caché.
- **Write-Through (escrituras, por defecto):** se escribe primero en MongoDB (commit) y luego se actualiza la caché – máxima consistencia.
- **Write-Behind (opcional, alto throughput):** se actualiza la caché y se difiere la escritura a disco vía worker; requiere WAL para no perder escrituras recientes ante caídas.
- **TTL:** todo objeto cacheado expira a los 300 segundos, garantizando invalidación automática y uso acotado de memoria.

5.2 Flujo de lectura (Cache-Aside)



Secuencia: (1) el cliente envía la petición; (2) el balanceador elige un pod de API; (3) la API calcula la clave de caché; (4) consulta DragonflyDB; (5) si hit, responde de inmediato; (6) si miss, consulta MongoDB; (7) guarda el documento con TTL de 300 s; (8) responde. Esto minimiza las lecturas a MongoDB manteniendo datos frescos hasta cinco minutos.

5.3 Flujo de escritura (Write-Through)



Secuencia: (1) validar la petición; (2) escribir en MongoDB; (3) tras el éxito, actualizar la caché; (4) aplicar TTL de 300 s; (5) responder 201. MongoDB permanece como fuente autoritativa mientras DragonflyDB refleja el último estado.

5.4 Expiración de caché

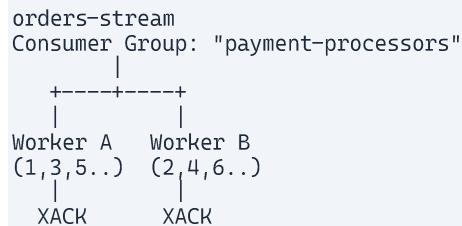
```
TTL = 300 segundos  
SET user:100 {json} EX 300
```

Ventajas: invalidación automática, uso de memoria acotado, menor riesgo de datos obsoletos y ausencia de trabajos de limpieza manual.

6 Arquitectura orientada a eventos – Streams

A diferencia del Pub/Sub clásico (fire-and-forget), los Streams son un log inmutable y persistente: los mensajes permanecen ordenados en el tiempo hasta confirmarse (XACK) y eliminarse (XDEL/XTRIM). Esto desacopla las tareas diferidas (pagos, correos, notificaciones, reportes) del hilo principal y protege a la base de datos ante picos de tráfico.

6.1 Consumer Groups – procesamiento distribuido



- Cada mensaje se asigna a exactamente un worker dentro del grupo (paralelización sin duplicados).
- Los mensajes no confirmados permanecen en la Pending Entry List (PEL).
- Si un worker cae, otro reclama sus mensajes pendientes con XCLAIM.
- Múltiples Consumer Groups independientes pueden leer el mismo Stream.
- Entrega at-least-once: los mensajes se retienen hasta la confirmación XACK.

6.2 Replay de eventos

Los Streams permiten reproducir eventos históricos desde cualquier punto del log – fundamental para resiliencia y auditoría:

- Recuperación tras caídas: reprocesar desde el último checkpoint.
- Recalcular métricas o agregaciones con nueva lógica de negocio.
- Auditoría: recorrer la línea de tiempo completa de eventos.
- Pruebas: reproducir escenarios reales en staging.

XRANGE lee por rango de IDs (timestamps); XREVRANGE en orden inverso; XREAD con ID «0-0» reproduce desde el inicio absoluto.

6.3 Comandos clave

XADD stream-name * campo valor	# Insertar evento
XREADGROUP GROUP g consumer BLOCK 0 STREAMS stream-name >	# Leer
XACK stream-name group message-id	# Confirmar
XCLAIM stream-name group consumer 0 message-id	# Reasignar
XRANGE stream-name - +	# Replay

7 Escalabilidad

El escalado opera en cada plano de forma independiente y sin rediseño de la aplicación.

Componente	Estrategia de escalado
API Gateway (Rust)	Aumentar réplicas sin estado (autoescalado)
DragonflyDB	Escalado vertical multi-core; añadir shards/réplicas en modo clúster
MongoDB	Añadir miembros al replica set o clústeres shardeados
Balanceador	Múltiples instancias

7.1 Replicación (escalado de lectura)

El nodo Primary acepta todas las escrituras; las réplicas absorben el tráfico de lectura masivo y garantizan continuidad ante fallos. Beneficios: read scaling, alta disponibilidad, failover rápido (milisegundos). Regla invariante: sólo el Primary acepta escrituras.

7.2 Sharding (escalado horizontal del espacio de datos)

En modo clúster, el espacio de claves se distribuye en 16 384 hash slots; la función CRC16 sobre la clave determina el slot destino.

```
Primary 1 -> Slots    0 - 5 460
Primary 2 -> Slots  5 461 - 10 922
Primary 3 -> Slots 10 923 - 16 383
```

- Escalabilidad horizontal: añadir nodos redistribuye slots automáticamente.
- Conjuntos de datos mayores que la RAM de un solo servidor.
- Throughput multiplicado por el número de shards.

SIMPLIFICACION CON DRAGONFLY Como dragonflydb es multinodo, un único nodo suele ofrecer el throughput que en Redis exigiría varios shards. El sharding pasa de ser obligatorio a ser una opción para cuando los datos exceden la memoria de un nodo.

7.3 Smart Clients – enrutamiento eficiente

- El cliente descubre y mantiene en memoria el mapa topológico (qué nodo posee qué rangos de slots).
- Calcula el hash slot de la clave y envía la consulta directamente al nodo correcto.
- Elimina redirecciones MOVED y ASK, reduciendo la latencia a un solo RTT.
- Ante failover/resharding, actualiza su mapa de forma transparente.

Bibliotecas de referencia: redis-rs (Rust), redis-py (Python), ioredis (Node.js), Jedis / Lettuce (Java).

8 Alta disponibilidad y resiliencia

8.1 Protocolo Gossip (heartbeats)

Los nodos intercambian mensajes periódicos de estado; cada nodo conoce el estado de los demás sin coordinador central. Provee detección de fallos distribuida y tolerante a particiones de red, propagación de cambios de topología en tiempo real y es la base del failover automático.

8.2 Failover automático

1. Nodos detectan fallo del Primary vía Gossip
2. La Replica asociada inicia elección de liderazgo
3. Quórum de nodos aprueba la promoción
4. Replica promovida a Primary
5. El clúster actualiza el enrutamiento interno
6. Los Smart Clients reciben la nueva topología
7. La aplicación continúa operando sin interrupción

8.3 Consistencia por quórum

Para datos transaccionales críticos puede configurarse escritura con quórum: la operación se confirma al cliente sólo cuando la mayoría de los nodos ha persistido la escritura en disco y en réplica – durabilidad máxima a costa de latencia marginal.

8.4 Escenarios de fallo

Fallo	Comportamiento del sistema
Instancia de API cae	El balanceador la expulsa; el tráfico continúa en las demás. Sin pérdida de datos por ser stateless.
Primary de caché cae	La réplica es promovida; el clúster actualiza topología; el Smart Client refresca su enrutamiento; la aplicación continúa.
Secundario de MongoDB cae	Lecturas y escrituras continúan; el replica set sólo elige nuevo primary si cae el actual.

9 Durabilidad y persistencia

Para un motor con millones de operaciones por segundo se requiere la velocidad de la RAM acoplada a una política de cero pérdida de datos. Se combinan tres mecanismos:

9.1 Persistencia híbrida AOF + RDB (capa de memoria)

Mecanismo	Características	rol en la arquitectura
AOF (Append Only File)	Registra cada escritura secuencialmente. Máxima durabilidad, mínima pérdida, replay al reiniciar.	Método primario en producción: ninguna escritura confirmada se pierde.
RDB (Snapshots)	Copias point-in-time compactas; recuperación rápida. Puede perder escrituras entre snapshots.	Respaldo estratégico: migraciones, DR y backups fríos externos.

```
# AOF -- método primario
appendonly yes
appendfsync everysec      # balance durabilidad/rendimiento

# RDB -- respaldo secundario
save 900 1                # snapshot si >=1 cambio en 15 min
save 300 10               # snapshot si >=10 cambios en 5 min
save 60 10000             # snapshot si >=10k cambios en 1 min
```

9.2 Write-Ahead Log (WAL) – transacciones

El WAL es una de las piezas más importantes para la recuperación ante caídas y debe implementarse antes que las estrategias de caché avanzadas (write-behind). Flujo de una transacción:

```
BEGIN
|
Write-Ahead Log      <- se registra la intención antes de aplicar
|
Modificar caché
|
Flush
|
Commit
|
Borrar WAL
```

9.3 Durabilidad de MongoDB

MongoDB aporta durabilidad nativa mediante su oplog (equivalente funcional a un WAL) y el journaling de WiredTiger, replicando cada operación a los secundarios del replica set. Es el ancla de durabilidad del sistema completo.

10 Seguridad – cifrado y control de acceso

10.1 Cifrado en reposo (AES-256-GCM)

Cada colección/archivo se cifra con AES-256-GCM (cifrado autenticado). El formato en disco separa un encabezado en claro de los datos cifrados:

```
users.db
  Header
  Version
  Salt
  Nonce
  --- todo lo siguiente cifrado con AES-256-GCM ---
  Encrypted Data
  Authentication Tag
```

10.2 Envelope encryption con Cloud KMS

Cada colección obtiene su propia Data Encryption Key (DEK). La DEK no se guarda en claro: se cifra con una clave maestra que reside en un Cloud KMS. Nunca se expone la clave maestra directamente.

Escritura: DEK AES-256-GCM Colección cifrada DEK -> cifrada por Master Key (almacenada en Cloud KMS)	Apertura de la colección: Leer DEK cifrada Pedir a KMS que la descifre Recibir DEK Descifrar colección
--	--

Opciones de KMS: AWS KMS, Google Cloud KMS, Azure Key Vault, HashiCorp Vault (self-hosted).

- Las claves nunca abandonan el KMS sin protección.
- Rotación de claves sencilla.
- Registro de accesos (access logging) y permisos de grano fino.

10.3 Índices sobre datos cifrados

Si los campos indexados permanecen en claro, la indexación es directa. Si están cifrados, se requiere cifrado determinista u otro enfoque de cifrado buscable, cada uno con sus compromisos. Esta es una decisión de diseño a resolver por caso de uso.

10.4 Controles adicionales

- **Contraseñas:** hash Argon2id – nunca cifradas ni en claro.
- **Transporte:** TLS para toda comunicación cliente-servidor.
- **Backups firmados** para detectar manipulación.
- **RBAC** (control de acceso basado en roles) y audit logging.

11 Workers en segundo plano y respaldos

11.1 Pool de workers

Un pool de trabajadores en segundo plano descarga al hilo principal de tareas costosas:

- Flush de caché y escritura diferida (write-behind).
- Cifrado de datos y escritura de colecciones.
- Construcción de índices.
- Eliminación de caché expirada y compactación de archivos.
- Respaldos y verificación de checksums.

11.2 Respaldos automáticos cifrados

Un planificador ejecuta periódicamente:

```
Pausar escrituras brevemente (o usar snapshot)
|
Crear snapshot
|
Comprimir (p. ej. Zstandard)
|
Cifrar backup
|
Subir a la nube
```

Ejemplo de disposición y política de retención:

```
backups/
  2026-01-01_00-00.backup
  2026-01-02_00-00.backup
  2026-01-03_00-00.backup

Retención:  Horarios x 24  |  Diarios x 30  |  Semanales x 12  |  Mensuales x 12
```

12 Índices

En lugar de escanear documentos completos, los índices mapean un valor de campo a un offset dentro del archivo (o, en MongoDB, a la ubicación del documento) mediante una estructura B+/B-tree.

```
Búsqueda:  username = "john"
|
B+ Tree
|
Offset
|
Leer documento
|
Descifrar
|
Devolver
```

MongoDB provee índices B-tree nativos (simples, compuestos, de texto, geoespaciales, TTL). Para colecciones cifradas con campos indexados sensibles, aplica la consideración de cifrado determinista de la §10.3.

13 Despliegue

13.1 Docker Compose (desarrollo / nodo único)

Para desarrollo local, DragonflyDB se levanta como un único servicio compatible con Redis, exponiendo el endpoint estándar:

```
services:
  dragonfly:
    image: docker.dragonflydb.io/dragonflydb/dragonfly
    container_name: dragonfly
    ulimits:
      memlock: -1
    ports:
      - "6379:6379"      # mismo puerto y protocolo que Redis
    command: --logtostderr
    volumes:
      - dragonfly-data:/data

  mongo:
    image: mongo:7
    container_name: mongo
    ports:
      - "27017:27017"
    volumes:
      - mongo-data:/data/db

volumes:
  dragonfly-data:
  mongo-data:
```

COMPATIBILIDAD Al ser drop-in de redis, si se prefiere conservar el modelo de cluster de 6 nodos + HAProxy de la fuente 2, la aplicación no cambia: sólo sustituye la imagen de servicio. El connection string `redis://127.0.0.1:6379` es idéntico.

13.2 Kubernetes (producción)

```
Namespace
|
+-- ingress-nginx
|
+-- api-gateway      (Deployment, replicas: 3)
|
+-- dragonfly        (StatefulSet; 1 nodo multi-core o N en cluster)
|
+-- mongod           (StatefulSet, 3 Pods: Primary + 2 Secondary)
|
+-- monitoring       (Prometheus + Grafana)
```

13.3 Cliente Rust (un solo connection string)

```
// Cargo.toml
// redis = { version = "0.32", features = ["tokio-comp"] }
// tokio = { version = "1", features = ["full"] }

use redis::{AsyncCommands, Client};

#[tokio::main]
async fn main() -> redis::RedisResult<()> {
    // UN SOLO CONNECTION STRING (DragonflyDB, compatible con Redis)
    let client = Client::open("redis://127.0.0.1:6379")?;
    let mut conn = client.get_multiplexed_async_connection().await?;
```

```
conn.set("usuario:1", "Carlos").await?;  
let nombre: String = conn.get("usuario:1").await?;  
println!("Usuario: {}", nombre);  
Ok::<>()  
}
```

La emisión de eventos a Streams desde Rust es idéntica a Redis:

```
redis::cmd("XADD")  
  .arg("orders").arg("*")  
  .arg("id").arg(1001)  
  .arg("cliente").arg("Juan")  
  .query_async:::<()>(&mut conn)  
  .await?;
```

14 Arquitectura de código unificada

Los tres módulos de la infraestructura de memoria (standalone, streams, cluster) se integran con la capa de MongoDB y el gateway en Rust en un único repositorio:

```
proyecto/
|-- gateway-rust/                # API sin estado (auth, lógica, drivers)
|   |-- src/
|   |   |-- main.rs
|   |   |-- cache.rs            # Smart Client DragonflyDB (Cache-Aside)
|   |   |-- repo_mongo.rs       # driver MongoDB (Write-Through)
|   |   |-- streams.rs          # XADD / Consumer Groups
|   |-- Cargo.toml
|-- storage/dragonfly/           # capa de memoria (Redis-compatible)
|   |-- docker-compose.yml      # standalone
|   |-- redis.conf              # AOF + RDB
|   |-- streams/                # producer / worker / replay
|   |-- cluster/                # 6 nodos + create-cluster.sh (opcional)
|   |-- scripts/                # consumer-group / reset / cleanup
|-- storage/mongo/              # sistema de registro
|   |-- replica-set/            # Primary + 2 Secondary
|   |-- indexes/                # definiciones de índices
|-- security/                   # envelope encryption + KMS
|   |-- kms.rs                  # DEK <=> Master Key (Cloud KMS)
|   |-- backup.sh               # snapshot -> comprimir -> cifrar -> nube
|-- k8s/                        # manifiestos de despliegue
|-- monitoring/                 # Prometheus + Grafana
```

Mapa: concepto de la fuente 4 -> implementación unificada

Concepto (motor de archivos, fuente 4)	Implementación en MongoDB + DragonflyDB
JSON collection files	Colecciones nativas de MongoDB (BSON/WiredTiger)
Caché write-behind con DragonflyDB	DragonflyDB como Cache-Aside/Write-Through (+ write-behind opcional con WAL)
AES-256-GCM + DEK por colección	Cifrado en reposo + CSFLE por campo/colección
Envelope encryption + Cloud KMS	Se conserva íntegro: DEK cifrada por master key en KMS
Indices B+ tree propios	Indices B-tree nativos de MongoDB
WAL para crash recovery	oplog + journaling de WiredTiger
Background workers	Pool de workers para backups, índices, flush, compactación

15 Características de rendimiento

Operación	Ruta
Cache Hit	Cliente -> Balanceador -> API -> DragonflyDB

Operacion	Ruta
Cache Miss	Cliente -> Balanceador -> API -> DragonflyDB -> MongoDB
Escritura	Cliente -> Balanceador -> API -> MongoDB -> DragonflyDB

Beneficios esperados:

- Carga de lectura de MongoDB significativamente reducida bajo patrones de acceso repetidos.
- Latencia baja para datos cacheados, servidos desde memoria multihilo.
- Escalabilidad horizontal por réplicas de API sin estado y sharding de la capa de memoria.
- Resiliencia por failover automático en la caché (Gossip) y en el replica set de MongoDB.

16 Hoja de ruta (roadmap)

Progresión práctica de capacidades hacia un motor que rivalice en arquitectura con bases documentales modernas (MongoDB, Couchbase, RavenDB), conservando la simplicidad operativa de la capa en memoria:

- ✓ Almacenamiento documental (colecciones y bases de datos)
- ✓ Operaciones CRUD
- ✓ Índices secundarios (B-tree / B+ tree)
- ✓ Cifrado AES-256-GCM
- ✓ Envelope encryption con Cloud KMS
- ✓ Write-Ahead Log (WAL)
- ✓ Pool de workers en segundo plano
- ✓ Caché DragonflyDB (Redis-compatible)
- ✓ Backups cifrados automáticos
- ✓ Compresión (p. ej. Zstandard)
- ✓ Transacciones
- ✓ Replicación
- ✓ Soporte de snapshots
- ✓ Planificador y optimizador de consultas
- ✓ Change streams / notificaciones de eventos
- ✓ Protocolos REST y binario nativo
- ✓ Sharding horizontal
- ✓ Clustering multi-nodo

17 Consideraciones de ingeniería y decisiones

Aspecto	Definición en la arquitectura unificada
Patrón de caché	Cache-Aside para lecturas; write-through para escrituras (write-Behind opcional).
TTL de caché	300 segundos (5 minutos).
Rol de DragonflyDB	Cache en memoria + Streams; reemplazo directo de Redis, no sistema de registro.
Rol de MongoDB	Fuente de verdad persistente y autoritativa.
Diseño de API	Servicios Rust sin estado para escalado horizontal.
Cliente de caché	Smart Client compatible con Redis; enruta directo al slot correcto.
Durabilidad	AOF+RDB en memoria; WAL/oplog + replica set en MongoDB.
Seguridad	AES-256-GCM, envelope encryption + KMS, Argon2id, TLS, RBAC, audit log.

Decisiones pendientes de validación

- **Modo de escritura por defecto:** Write-Through (consistencia) vs Write-Behind (throughput con WAL).
- **Topología de la capa de memoria:** nodo único DragonflyDB multi-core vs modo clúster emulado / 6 nodos + HAProxy.
- **Estrategia de índices cifrados:** cifrado determinista vs mantener campos indexados en claro.
- **Frontera funcional:** qué datos viven sólo en caché con TTL frente a los que exigen persistencia inmediata en MongoDB.

18 Conclusión

La arquitectura propuesta combina un API Gateway en Rust sin estado, una capa de memoria DragonflyDB –Redis con esteroides, multihilo y compatible drop-in– y un MongoDB Replica Set como sistema de registro, elevados con envelope encryption sobre Cloud KMS, WAL, persistencia híbrida AOF+RDB y respaldos cifrados. Empleando Cache-Aside con TTL de cinco minutos y sincronizando las escrituras a través de MongoDB antes de actualizar la caché, el diseño minimiza la carga de base de datos manteniendo la consistencia.

El resultado es un motor de datos rápido, potente y preparado para grandes cargas: latencia de memoria multihilo, throughput de millones de QPS, failover automático en milisegundos, durabilidad de nivel empresarial y seguridad de datos en reposo y en tránsito – todo expuesto a la aplicación tras un único connection string.